

✓ 들어가며

LangGraph는 복잡한 AI 에이전트 흐름을 제어하는 상태 기반 프레임워크입니다.

순환 구조와 조건 분기를 포함한 그래프 형태의 워크플로우로 에이전트를 구성하며, LangChain 컴포넌트와 완벽하게 통합되어 기존 생태계를 그대로 활용할 수 있습니다.

복잡하고 동적인 의사결정 프로세스 구현에 최적화되어 있으나, 개념 난이도가 높아 단순한 작업에는 오히려 비효율적일 수 있습니다.

주로 복잡한 의사결정 프로세스를 자동화하거나, 반복적 수정 및 검토가 필요한 환경, 장기 기억을 활용하는 대화형 시스템에 많이 사용됩니다.

LangGraph는 상태(State), 노드(Node), 엣지(Edge) 세 가지 핵심 요소로 구성됩니다.

이 세 요소가 서로 연결되어 에이전트의 전체 흐름을 만들어냅니다.

쉽게 이해하려면 지하철 노선도를 떠올려보세요.

상태는 현재 열차의 위치와 승객 정보, 노드는 각 역에서 수행되는 작업, 엣지는 역과 역을 잇는 노선입니다.

열차는 상황에 따라 다른 노선으로 갈아탈 수 있고, 같은 역을 다시 지날 수도 있습니다.

```
class State(TypedDict):
    messages : Annotated[list, add_messages] # History
    code : Annotated[Document, "Retrieve Response"]

class State(TypedDict):
    messages : Annotated[list, add_messages] # History
    code : Annotated[str, "Python code"]
    code_result : Annotated[str, "Python code Result"]
```

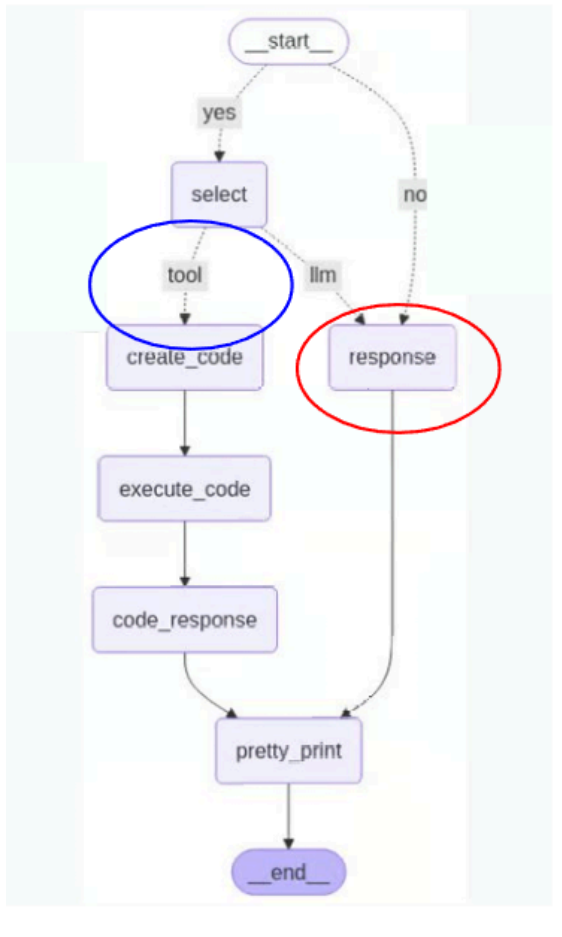
상태(State)는 에이전트가 현재 어느 단계에 있는지를 나타내는 정보입니다.

사용자의 입력, 지금까지의 도구 실행 결과, 다음 작업 지시 등이 모두 상태에 담깁니다.

상태는 작업이 진행되면서 지속적으로 업데이트되며, 전체 흐름의 컨텍스트 역할을 합니다.

상태는 단순한 변수가 아니라, 에이전트가 "지금 무엇을 알고 있는가"를 담은 공유 메모리입니다.

모든 노드는 이 상태를 읽고 쓰기 때문에, 상태 설계가 곧 전체 시스템 설계의 출발점이 됩니다.



노드(Node, 그림의 빨간 원)는 하나의 독립적인 작업 단위입니다.

각 노드는 현재 상태를 입력받아 정해진 작업을 수행한 뒤, 변경된 상태를 반환합니다.

예를 들어 "검색 실행", "답변 생성", "결과 검토"와 같은 개별 단계가 각각 하나의 노드가 됩니다.

노드는 일반 Python 함수로 작성되며, LLM 호출, 도구 실행, 조건 판단 등 어떤 로직도 담을 수 있습니다.

하나의 노드가 너무 많은 역할을 맡으면 재사용과 디버깅이 어려워지므로, 단일 책임 원칙에 따라 작게 나누는 것이 좋습니다.

엣지(Edge)는 노드 간의 흐름을 정의하는 연결선입니다.

작업 결과나 상태 조건에 따라 다음에 실행할 노드를 선택하며, 분기·반복·종료와 같은 흐름 제어가 가능합니다.

조건부 엣지를 활용하면 상황에 따라 서로 다른 경로로 흐름을 유도할 수 있어, LangGraph의 동적인 제어가 가능해집니다.

정리하면, LangGraph에서 에이전트는 상태를 공유하며 노드를 순서대로 실행하고, 엣지가 그 순서를 동적으로 결정합니다.

단순한 순차 실행을 넘어 조건 분기와 반복이 가능하기 때문에, 실제 업무 수준의 복잡한 AI 워크플로우를 구현할 수 있습니다.

LangGraph 기반 데이터 분석 에이전트를 구축하는 간단한 실습을 진행해보겠습니다

- State → Node → Edge 구조로 워크플로우 설계
- 타이타닉 / 올림픽 데이터셋을 활용한 분석 실습
 - history_node → select → code_executor → code_response / response 흐름

✓ 1. 패키지 설정

```
langchain-openai langchain-core langchain-community langchain-experimental fpdf2 pdfplumber request
```

```

===== 81.0/81.0 kB 2.1 MB/s eta 0:00:00
===== 43.6/43.6 kB 2.4 MB/s eta 0:00:00
===== 68.1/68.1 kB 2.3 MB/s eta 0:00:00
===== 87.7/87.7 kB 6.1 MB/s eta 0:00:00
===== 2.5/2.5 MB 51.9 MB/s eta 0:00:00
===== 210.1/210.1 kB 15.5 MB/s eta 0:00:00
===== 327.1/327.1 kB 21.7 MB/s eta 0:00:00
  
```

```

60.0/60.0 kB 3.9 MB/s eta 0:00:00
6.6/6.6 MB 47.7 MB/s eta 0:00:00
64.7/64.7 kB 4.4 MB/s eta 0:00:00
1.0/1.0 MB 46.8 MB/s eta 0:00:00
3.6/3.6 MB 61.8 MB/s eta 0:00:00
51.0/51.0 kB 3.7 MB/s eta 0:00:00
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed
google-colab 1.0.0 requires requests==2.32.4, but you have requests 2.32.5 which is incompatible.

```

2. 기본 import 및 API key 설정

```

import os
import re
import random
import warnings
import requests
import pdfplumber

from typing import Annotated, Literal, Optional
from typing_extensions import TypedDict

from pydantic import BaseModel, Field
from fpdf import FPDF

from langchain_openai import ChatOpenAI
from langchain_core.tools import tool
from langchain_core.messages import AIMessage, HumanMessage, ToolMessage
from langchain_core.prompts import PromptTemplate
from langchain_core.runnables import RunnableConfig

from langchain_community.tools.tavily_search import TavilySearchResults
from langchain_community.agent_toolkits import FileManagementToolkit
from langchain_experimental.tools.python.tool import PythonAstREPLTool

from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langgraph.prebuilt import ToolNode
from langgraph.checkpoint.memory import MemorySaver

warnings.filterwarnings("ignore")

```

```

# Colab 환경이면 사용
from google.colab import userdata

import os

os.environ["OPENAI_API_KEY"] = ""
os.environ["TAVILY_API_KEY"] = ""

if not os.environ.get("OPENAI_API_KEY"):
    raise ValueError("OPENAI_API_KEY가 설정되지 않았습니다.")

if not os.environ.get("TAVILY_API_KEY"):
    raise ValueError("TAVILY_API_KEY가 설정되지 않았습니다.")

```

3. 상태(State) 정의

```

class State(TypedDict):
    query: Annotated[str, "사용자 질문"]
    answer: Annotated[str, "최종 답변"]
    messages: Annotated[list, add_messages]
    tool_call: Annotated[list, "도구 호출 기록"]

```

4. LLM 정의

```
llm = ChatOpenAI(
    model="gpt-4o-mini",
    temperature=0
)
```

✓ 5. 사용자 정의 도구

5-1. PDF 읽기 도구

```
@tool
def read_pdf(file_path: str) -> str:
    """
    PDF 파일에서 텍스트를 추출합니다.
    예: './output.pdf'
    """
    try:
        text = ""
        with pdfplumber.open(file_path) as pdf:
            for page in pdf.pages:
                page_text = page.extract_text()
                if page_text:
                    text += page_text + "\n"
        return text.strip() if text.strip() else "PDF에서 텍스트를 추출할 수 없습니다."
    except Exception as e:
        return f"PDF 읽기 오류: {str(e)}"
```

5-2. PDF 저장 도구

```
@tool
def write_pdf(content: str, filename: str = "output.pdf", summary: bool = True) -> str:
    """
    텍스트를 PDF로 저장합니다.
    summary=True면 내용을 한 번 더 정리해서 보고서 형태로 다듬습니다.
    """
    try:
        final_content = content

        if summary:
            prompt = PromptTemplate.from_template("""
당신은 보고서 작성 어시스턴트입니다.
주어진 내용을 바탕으로 한국어 보고서를 작성하세요.

반드시 아래 구조를 따르세요.
1. 제목
2. 개요
3. 핵심 내용
4. 장점/특징
5. 한계 또는 유의사항
6. 인사이트

내용:
{content}
""")

            chain = prompt | llm
            final_content = chain.invoke({"content": content}).content

        os.makedirs("./fonts", exist_ok=True)
        font_path = "./fonts/NotoSansKR-Regular.ttf"

        if not os.path.exists(font_path):
            font_url = "https://github.com/google/fonts/raw/main/ofl/notosanskr/NotoSansKR%5Bwght%5D"
            response = requests.get(font_url, timeout=20)
            response.raise_for_status()
            with open(font_path, "wb") as f:
                f.write(response.content)

        pdf = FPDF()
        pdf.set_auto_page_break(auto=True, margin=15)
        pdf.add_page()
```

```

pdf.add_font("NotoSansKR", "", font_path, uni=True)
pdf.set_font("NotoSansKR", size=12)

for line in final_content.split("\n"):
    line = line.strip()
    if not line:
        pdf.ln(4)
    else:
        pdf.multi_cell(0, 8, line)

pdf.output(filename)
return f"{filename} 저장 완료"

except Exception as e:
    return f"PDF 저장 오류: {str(e)}"

```

5-3. 파이썬 코드 실행 도구

```
python_tool = PythonAstREPLTool()
```

5-4. 파일 관리 도구

```

file_tools = FileManagementToolkit(
    selected_tools=["file_delete", "list_directory"]
).get_tools()

listdir_tool = [t for t in file_tools if t.name == "list_directory"][0]
delete_tool = [t for t in file_tools if t.name == "file_delete"][0]

```

5-5. 웹 검색 도구

```
search_tool = TavilySearchResults(max_results=5)
```

✓ 6. 전체 도구 묶기

```

tools = [search_tool, python_tool, write_pdf, read_pdf, listdir_tool, delete_tool]
tool_node = ToolNode(tools)
llm_with_tools = llm.bind_tools(tools)

```

7. 보조 함수들

7-1. 최근 대화 기억

```

def shorterm_memory(state: State):
    if len(state["messages"]) > 8:
        history = state["messages"][-8:-1]
    elif len(state["messages"]) == 1:
        history = []
    else:
        history = state["messages"][:-1]
    return history

```

7-2. 코드 블록 추출 함수

```

def extract_python_code(text: str) -> Optional[str]:
    pattern = r"```python\s*(.*?)```"
    match = re.search(pattern, text, re.DOTALL | re.IGNORECASE)
    if match:
        return match.group(1).strip()
    return None

```

8. 구조화 출력용 분류기

8-1. 히스토리만으로 답변 가능한지 판단

```
class HistoryChecker(BaseModel):
    yes_no: Literal["yes", "no"] = Field(
        ...,
        description='이전 대화만으로 답변 가능한지 "yes", 아니면 "no"'
    )

history_checker = llm.with_structured_output(HistoryChecker)
```

```
def history_check(state: State):
    if len(state["messages"]) <= 1:
        return "no"

    prompt = PromptTemplate.from_template("""
이전 대화 기록만 보고 질문에 답변 가능한지 판단하세요.
가능하면 "yes", 불가능하면 "no"를 반환하세요.

대화 기록:
{history}

질문:
{query}
""")
    chain = prompt | history_checker
    history = shortterm_memory(state)
    result = chain.invoke({
        "history": history,
        "query": state["query"]
    })
    return result.yes_no
```

8-2. 답변이 충분한지 판단

```
class AnswerChecker(BaseModel):
    end: Literal["end", "tool"] = Field(
        ...,
        description='질문이 해결되면 "end", 도구가 더 필요하면 "tool"'
    )

answer_checker = llm.with_structured_output(AnswerChecker)
```

```
def answer_check(state: State):
    prompt = PromptTemplate.from_template("""
당신은 답변 품질 판별기입니다.

질문:
{query}

현재 답변:
{answer}

최근 대화:
{history}

현재 답변만으로 사용자의 요청이 충분히 해결되었으면 "end",
아직 웹검색/파일저장/코드실행 등의 도구가 더 필요하면 "tool"을 반환하세요.
""")
    chain = prompt | answer_checker
    history = shortterm_memory(state)

    answer_text = state["answer"]
    if not isinstance(answer_text, str):
        answer_text = str(answer_text)

    result = chain.invoke({
```

```

        "history": history,
        "answer": answer_text,
        "query": state["query"]
    })
    return result.end

```

✓ 9) 노드 함수

9-1. history_node

```

def history_node(state: State):
    if len(state["messages"]) == 1:
        return {
            "answer": "초기 상태",
            "tool_call": []
        }
    return state

```

9-2. memory_chat

```

def memory_chat(state: State):
    prompt = PromptTemplate.from_template("""
이전 대화 기록을 참고해 질문에 답변하세요.
기록만으로 충분하지 않다면 일반적인 내부 지식으로 답변하세요.

대화 기록:
{history}

질문:
{query}
""")
    chain = prompt | llm
    history = shortterm_memory(state)
    result = chain.invoke({
        "history": history,
        "query": state["query"]
    })
    return {
        "answer": result.content,
        "messages": result
    }

```

9-3. select

```

def select(state: State):
    user_query = state["query"]

    # 1) 코드 블록이 있으면 바로 python 도구 실행 유도
    extracted_code = extract_python_code(user_query)
    if extracted_code:
        tool_call_message = AIMessage(
            content="파이썬 코드를 실행합니다.",
            tool_calls=[
                {
                    "name": "python_repl_ast",
                    "args": {"query": extracted_code},
                    "id": "direct_python_call",
                    "type": "tool_call"
                }
            ]
        )
    return {
        "messages": tool_call_message,
        "tool_call": tool_call_message.tool_calls
    }

prompt = PromptTemplate.from_template("""

```

당신은 도구 선택 에이전트입니다.
사용 가능한 도구는 다음과 같습니다.

1. tavily_search_results_json: 웹 검색
2. python_repl_ast: 파이썬 코드 실행
3. write_pdf: PDF 파일 저장
4. read_pdf: PDF 읽기
5. list_directory: 현재 폴더 파일 확인
6. file_delete: 파일 삭제

규칙:

- "조사", "정리", "검색", "최신 정보" 요청이면 웹 검색을 우선 사용하세요.
- "pdf로 저장", "레포트로 저장", "보고서 파일로 저장" 요청이면 write_pdf를 사용하세요.
- 사용자가 ``python ... `` 코드블록을 주면 python_repl_ast를 사용하세요.
- 파일이 저장됐는지 확인 요청이면 list_directory를 사용하세요.
- 저장된 pdf 내용을 확인해달라고 하면 read_pdf를 사용하세요.
- 삭제 요청이면 file_delete를 사용하세요.
- 필요한 경우 여러 도구를 순차적으로 사용할 수 있습니다.

최근 대화:

{history}

현재 질문:

{query}

""")

```
chain = prompt | llm_with_tools
history = shortterm_memory(state)
result = chain.invoke({
    "history": history,
    "query": user_query
})

if hasattr(result, "tool_calls") and len(result.tool_calls) > 0:
    return {
        "messages": result,
        "tool_call": result.tool_calls
    }
else:
    fallback = AIMessage(content="도구 선택 실패. 일반 답변으로 전환합니다.")
    return {
        "messages": fallback,
        "answer": fallback.content,
        "tool_call": []
    }
```

9-4. response

```
def response(state: State):
    last_message = state["messages"][-1]

    if isinstance(last_message, ToolMessage):
        return {
            "answer": last_message.content
        }

    if isinstance(last_message, AIMessage):
        return {
            "answer": last_message.content
        }

    return {
        "answer": str(last_message)
    }
```

10. 그래프 구성

```
graph_builder = StateGraph(State)

graph_builder.add_node("history_node", history_node)
graph_builder.add_node("memory_chat", memory_chat)
```



```

graph_builder.add_node("select", select)
graph_builder.add_node("tools", tool_node)
graph_builder.add_node("response", response)

graph_builder.add_edge(START, "history_node")

graph_builder.add_conditional_edges(
    "history_node",
    history_check,
    {
        "yes": "memory_chat",
        "no": "select"
    }
)

graph_builder.add_edge("select", "tools")
graph_builder.add_edge("tools", "response")
graph_builder.add_edge("memory_chat", "response")

graph_builder.add_conditional_edges(
    "response",
    answer_check,
    {
        "end": END,
        "tool": "select"
    }
)

memory = MemorySaver()
graph = graph_builder.compile(checkpointer=memory)

```

11. 실행 설정 함수

```

def reset_config(limit=15):
    thread_id = random.randint(1, 999999)
    config = RunnableConfig(
        recursion_limit=limit,
        configurable={"thread_id": thread_id}
    )
    return config

```

12. 스트리밍 함수

```

def streaming(query, config, mode="values"):
    result = graph.stream(
        {
            "messages": [HumanMessage(content=query)],
            "query": query,
            "answer": "",
            "tool_call": []
        },
        config=config,
        stream_mode=mode
    )

    final_answer = None

    if mode == "values":
        for step in result:
            if "messages" in step and len(step["messages"]) > 0:
                last_msg = step["messages"][-1]
                try:
                    last_msg.pretty_print()
                except:
                    print(last_msg)
            if "answer" in step:
                final_answer = step["answer"]

    elif mode == "updates":
        for step in result:
            print(step)

```

```
return final_answer
```

✓ 실행 예시 1: 보고서 PDF 만들기

```
config = reset_config()

query = """
모두의연구소는 어떤 곳인지 조사해서 한국어 보고서로 정리해줘.
다음 항목을 포함해줘.
1. 기관 소개
2. 주요 교육/커뮤니티 활동
3. 특징
4. 장점
5. 한계 또는 주의점
6. 인사이트

마지막 결과는 PDF 파일로 저장해줘.
파일명은 "모두의연구소_레포트.pdf"로 해줘.
"""

streaming(query, config)
```

===== Human Message =====

모두의연구소는 어떤 곳인지 조사해서 한국어 보고서로 정리해줘.
다음 항목을 포함해줘.

1. 기관 소개
2. 주요 교육/커뮤니티 활동
3. 특징
4. 장점
5. 한계 또는 주의점
6. 인사이트

마지막 결과는 PDF 파일로 저장해줘.
파일명은 "모두의연구소_레포트.pdf"로 해줘.

===== Human Message =====

모두의연구소는 어떤 곳인지 조사해서 한국어 보고서로 정리해줘.
다음 항목을 포함해줘.

1. 기관 소개
2. 주요 교육/커뮤니티 활동
3. 특징
4. 장점
5. 한계 또는 주의점
6. 인사이트

마지막 결과는 PDF 파일로 저장해줘.
파일명은 "모두의연구소_레포트.pdf"로 해줘.

```
-----
RateLimitError                                Traceback (most recent call last)
/tmp/ipykernel_4187/2075726980.py in <cell line: 0>()
    15 """
    16
--> 17 streaming(query, config)
```

```
----- 19 frames -----
/usr/local/lib/python3.12/dist-packages/openai/_base_client.py in request(self, cast_to, options,
stream, stream_cls)
    1068
    1069         log.debug("Re-raising status error")
-> 1070         raise self._make_status_error_from_response(err.response) from None
    1071
    1072         break
```

```
RateLimitError: Error code: 429 - {'error': {'message': 'You exceeded your current quota, please
check your plan and billing details. For more information on this error, read the docs:
https://platform.openai.com/docs/guides/error-codes/api-errors.', 'type': 'insufficient_quota',
'param': None, 'code': 'insufficient_quota'}}
```

다음 단계: 오류 설명

실행 예시 2: 저장된 파일 확인

```
config = reset_config()

query = "현재 폴더에 어떤 파일이 있는지 보여줘."
streaming(query, config)
```

실행 예시 3: PDF 내용 확인

```
config = reset_config()

query = '"모두의연구소_레포트.pdf" 내용을 읽어서 핵심만 5줄로 요약해줘.'
streaming(query, config)
```

실행 예시 4: 코드 실행 기능 시연

```
config = reset_config()

query = """
아래 코드 실행시켜주세요.
```python
result = 0
for i in range(1, 21):
 print(f"{i}번째 출력:", i)
 result += i
print("최종 결과:", result)
```

## ✓ 실행 예시 5: 제출용으로 주제 바꾼 버전

아래처럼 바꾸면 “예시 주제를 바꿔 구현” 조건에도 맞는다.

```
config = reset_config()

query = """
모두의 연구소의 AI/개발 교육 특징을 비교 조사.
다음 형식으로 정리해줘.
1. 기관별 소개
2. 강의/부트캠프 특징
3. 대상자
4. 장단점 비교
5. 추천 대상
6. 최종 인사이트

결과는 PDF 파일로 저장.
파일명은 "교육기관비교_레포트.pdf"로 지정.
"""

streaming(query, config)
```

```
print("프로젝트 주제: AI 교육기관 조사 보고서 생성 에이전트")
print("사용 툴: TavilySearchResults, PythonAstREPLTool, write_pdf, read_pdf, FileManagementToolkit")
print("최종 산출물: PDF 보고서 자동 생성")
```

## ✓ GENAI\_DAY5\_AgenticAI\_Code\_1\_Langgraph\_DataAgent

# 데이터프레임 불러오기

```
url = "/content/titanic.csv"
df = pd.read_csv(url)
```

# 데이터프레임 정보 확인

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
Column Non-Null Count Dtype
--- -
0 PassengerId 891 non-null int64
1 Survived 891 non-null int64
2 Pclass 891 non-null int64
3 Name 891 non-null object
4 Sex 891 non-null object
5 Age 714 non-null float64
6 SibSp 891 non-null int64
7 Parch 891 non-null int64
8 Ticket 891 non-null object
9 Fare 891 non-null float64
10 Cabin 204 non-null object
11 Embarked 889 non-null object
```

```
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

## 1. State 정의

LangGraph의 핵심인 State 클래스를 정의합니다. 이 클래스는 에이전트가 실행되는 동안 필요한 모든 정보를 저장합니다:

- `messages`: 대화 히스토리
- `code`: 실행할 Python 코드
- `code_result`: 코드 실행 결과

```
그래프 상태 정의

class State(TypedDict):
 messages : Annotated[list, add_messages] # History
 code : Annotated[str, "Python code"]
 code_result : Annotated[str, "Python code Result"]
```

ChatGPT 모델을 설정하고, Python 코드 실행 도구를 연결합니다.

temperature=0은 일관된 응답을 위해 창의성을 최소화합니다.

도구(Tool)란 LLM이 텍스트 생성 외에 **외부 기능을 실행**할 수 있도록 연결해주는 모듈입니다.

예를 들어 계산, 코드 실행, 검색 등의 작업을 LLM 스스로 판단하여 호출할 수 있습니다.

LLM 단독으로는 코드를 실행할 수 없습니다.

이후 `bind_tools()`를 사용하면 LLM이 필요할 때 직접 도구를 호출할 수 있게 됩니다.

여기서 정의한 `PythonAstREPLTool`는, LLM이 파이썬 코드를 직접 실행하고 결과를 받아올 수 있게 해주는 도구입니다.

```
LLM 로드

llm = ChatOpenAI(model="gpt-4o-mini",
 temperature=0.)

llm = ChatGoogleGenerativeAI(model="gemini-2.5-flash-preview-04-17",
temperature=0.,
convert_system_message_to_human=True,
)

파이썬 코드 실행 도구 정의
tool = PythonAstREPLTool(name="python_repl_ast",
 description="A Python shell. Use this to execute python commands. \
Input should be a valid python command. When using this tool, \
sometimes output is abbreviated - make sure it does not look at \
locals={\"df\":df})
```

```
tool.invoke("""python for i in range(10): print(i)""")
```

```
'0\n1\n2\n3\n4\n5\n6\n7\n8\n9\n'
```

## 1-1 Langchain

기존 랭체인에서의 활용방법을 비교하기 위해 정의된 세션입니다.

랭그래프로 같은 동작을 하면 어떻게 달라지는지 비교해보세요!

아래 코드는 데이터프레임(df)을 받아서 LLM(AI)에게 전달하고, 데이터셋에 어울리는 **제목**과 **요약**을 자동으로 만들어주는 함수입니다.

. 데이터가 3000행보다 많으면 3000개만 샘플링합니다. AI에게 너무 많은 데이터를 한번에 보내면 비효율적이기 때문입니다.

2. 프롬프트 템플릿에 데이터를 넣어 AI에게 제목/요약 생성을 요청합니다.

3. AI의 응답에서 제목과 요약 텍스트만 추출합니다.

4. 응답 형식이 잘못됐을 경우 기본값("Untitled", "No Summary") 반환합니다.

```
데이터셋 제목, 요약 생성 함수
```

```
def create_title_summary(df:pd.DataFrame) -> Tuple[str, str]:

 # 데이터셋이 너무 클 시 3000개의 샘플만 제공
 df_sampled = df.sample(n=3000) if len(df) > 3000 else df

 prompt = PromptTemplate.from_template("""
 당신은 요약 전문가입니다.

 데이터셋 : {df}

 데이터셋의 정보를 보고 제목과 요약을 만들어냅니다.
 제목은 이 데이터셋을 가장 잘 표현할 수 있는 제목으로 결정하여야합니다.

 제목:
 요약:

 """)

 chain = prompt | llm

 # chain.invoke는 AI Message객체를 반환함. 그 중 str 형태인 content만 추출
 result = chain.invoke({"df":df_sampled})

 title = "Untitled"
 summary = "No Summary"

 # 요약 양식에 맞지 않는 답변을 할 시 "Untitled"과 "No Summary"로 설정하도록 예외처리
 try:
 content = result.content
 content = content.replace("## 결과:\n\n", "")
 lines = content.split("\n")
 title = lines[0].replace("## 제목:", "").strip()
 summary = "\n".join(lines[1:]).replace("## 요약:", "").strip()
 except:
 pass

 print("===== 제목, 요약 생성 완료 =====")

 return title, summary
```

```
제목, 요약 생성
```

```
title, summary = create_title_summary(df)
```

```
===== 제목, 요약 생성 완료 =====
```

```
print("제목 : ", title)
print("요약 : ", summary)
```

```
제목 : **제목:** 타이타닉 승객 생존 데이터셋
```

```
요약 : **요약:** 이 데이터셋은 타이타닉 호의 승객에 대한 정보를 포함하고 있으며, 총 891명의 승객 데이터가 기록되어 있습니다.
```

LLM에 도구를 쥐어줍니다.

```
LLM에게 도구 할당
```

```
llm_with_tools = llm.bind_tools([tool])
```

```
llm_with_tools.invoke("1부터 10까지 출력하는 파이썬 코드")
```

```
AIMessage(content='', additional_kwargs={'refusal': None}, response_metadata={'token_usage':
{'completion_tokens': 31, 'prompt_tokens': 103, 'total_tokens': 134, 'completion_tokens_details':
{'accepted_prediction_tokens': 0, 'audio_tokens': 0, 'reasoning_tokens': 0,
'rejected_prediction_tokens': 0}, 'prompt_tokens_details': {'audio_tokens': 0, 'cached_tokens':
0}}, 'model_provider': 'openai', 'model_name': 'gpt-4o-mini-2024-07-18', 'system_fingerprint':
'fp_80d7a26d20', 'id': 'chatcmpl-DKgS7fTzVe79Xtpp87yXTxjgfn6AU', 'service_tier': 'default',
'finish_reason': 'tool_calls', 'logprobs': None}, id='lc_run--019d0009-8e01-7580-8566-8fb7feff67e1-
0', tool_calls=[{'name': 'python_repl_ast', 'args': {'query': 'for i in range(1, 11):\n
print(i)'}, 'id': 'call_SxyqbevMTfCpvvb4QBobG79A', 'type': 'tool_call'}], invalid_tool_calls=[],
```

```
usage_metadata={'input_tokens': 103, 'output_tokens': 31, 'total_tokens': 134,
'input_token_details': {'audio': 0, 'cache_read': 0}, 'output_token_details': {'audio': 0,
'reasoning': 0}})
```

```
class HistoryChecker(BaseModel):
 """
 이전의 대화 기록을 참고하여 질문에 대해 답변할 수 있는지 판단합니다.
 답변할 수 있다면 "yes", 답변할 수 없다면 "no"를 반환합니다.
 """

 yes_no : Literal["yes", "no"] = Field(..., description="""Use your previous conversation history to answer the question.
 Return "yes" if you can answer, "no" if you can't answer.""")
```

```
LLM의 응답을 HistoryChecker 클래스 구조에 맞춰 파싱하도록 설정
```

```
history_checker = llm.with_structured_output(HistoryChecker)
```

```
history_checker.invoke("오늘 인천 날씨는 어때?")
```

```
HistoryChecker(yes_no='no')
```

```
히스토리 기반 답변 분기를 위한 함수 설정
```

```
def history_check(state:State):

 # 이전 대화가 없으면 무조건 "no" 반환
 if len(state["messages"]) <= 1:
 return "no"

 prompt = PromptTemplate.from_template("""

 이전의 대화 기록을 참고하여 질문에 대해 답변할 수 있는지 판단합니다.
 답변할 수 있다면 "yes", 답변할 수 없다면 "no"를 반환합니다.

 대화 기록 : {history}

 질문 : {query}

 """)

 chain = prompt | history_checker

 result = chain.invoke({"history":state["messages"][:-1],
 "query":state["messages"][-1]})

 return result.yes_no
```

## ✓ 2. 워크플로우 노드 함수들

각 노드는 특정 작업을 수행합니다:

- `history_node`: 이전 대화 확인
- `select`: 코드 실행 여부 결정
- `code_executor`: Python 코드 실행
- `code_response`: 결과 정리 및 응답

```
식별을 위한 노드
```

```
def history_node(state:State):

 return
```

```
도구 사용 여부 결정 노드
```

```
def select(state:State):
 prompt = PromptTemplate.from_template("""
 당신은 데이터 분석가입니다.
 당신은 데이터프레임인 df를 활용할 수 있습니다.
```

당신이 활용 가능한 데이터프레임의 예시는 아래와 같습니다.  
 아래의 예시는 `print(df.head())`의 실행결과입니다.  
 활용 가능한 데이터프레임을 갖고 있으니 데이터프레임을 새로 생성할 필요 없습니다.  
 코드에 한글이 필요하다면 `import koreanize\_matplotlib`을 사용하세요.

당신은 'python\_repl\_ast'라는 파이썬 코드 실행 도구만을 가지고 있습니다.  
 아래의 내용을 참고하여 질문에 대한 코드를 생성합니다.

```
df : {df}

title : {title}
summary : {summary}

query : {query}
"""
)

chain = prompt | llm_with_tools

result = chain.invoke({"df": df.head().to_string(), #df.head(),
 "title":title,
 "summary":summary,
 "query":state["messages"][-1].content})

if hasattr(result, "tool_calls") and len(result.tool_calls) > 0:
 tool_calls = result.tool_calls

 # 코드 추출
 code = tool_calls[0]["args"]["query"]
 return {"code": code}
else:
 # 도구 사용을 안한다면 코드는 공백으로 설정
 return {"code":""}
```

# 코드 실행 노드

```
def code_executor(state:State):

 try:
 # 코드 실행 결과인 이미지 추출
 if "plt" in state["code"] or "sns" in state["code"]:
 save_fig = ""

 import io
 import base64

 buf = io.BytesIO()
 plt.savefig(buf, format="png")
 buf.seek(0)

 encoded = base64.b64encode(buf.read()).decode("utf-8")

 print(encoded)
 """

 execute_code = state["code"] + save_fig

 result = tool.invoke(execute_code)

 return {"code_result":result}
 # 시각화가 아닌 경우 코드만 실행
 else:
 result = tool.invoke(state["code"])
 return {"code_result":str(result)}
 except:
 return {"code_result": ""}
```

# 코드에 의한 답변 노드

```
def code_response(state:State):
 prompt = ChatPromptTemplate.from_messages([
 ("system", """코드 : {code} \n
 결과 : {code_result}
```



```

당신은 주어진 코드와 코드 결과를 바탕으로 질의에 대해 답변합니다.
절대 코드에 대해 설명하지마세요.
독자는 프로그래머가 아닙니다.
항상 출력되는 값을 기준으로 설명합니다.
숫자가 매우 중요합니다. 숫자에 대한 정보를 잊지 마세요.
데이터 분석과 관련된 코드가 입력된다면 항상 인사이트를 포함하세요. """),
("human", '{query}')
])

chain = prompt | llm

result = chain.invoke({"code":state["code"],
 "code_result":state["code_result"],
 "query":state["messages"][-1]})

return { "messages": result}

```

# 기억 및 일반 응답 노드

```

def response(state:State):
 prompt = PromptTemplate.from_template("""

 이전의 대화 기록을 참고하여 질문에 대해 답변하세요.
 아래 대화 기록을 첨부합니다.
 대화 기록을 통해 답변이 어렵다면 내부 지식을 참조하세요.

 대화 기록 : {history}

 질문 : {query}

 """)

 chain = prompt | llm

 result = chain.invoke({"history":state["messages"][:-1],
 "query":state["messages"][-1]})

 return {"#answer":result.content,
 "messages":result}

```

### ✓ 3. 워크플로우 그래프 구축

StateGraph를 사용해 노드들을 연결하고 실행 흐름을 정의합니다. 조건부 엣지를 통해 상황에 따라 다른 경로로 분기할 수 있습니다.

# 그래프 정의

```
graph_builder = StateGraph(State)
```

# 노드 및 엣지 정의

```

graph_builder.add_node("history_node", history_node)
graph_builder.add_node("select", select)
graph_builder.add_node("code_executor", code_executor)
graph_builder.add_node("code_response", code_response)
graph_builder.add_node("response", response)

graph_builder.add_edge(START, "history_node")
graph_builder.add_conditional_edges("history_node",
 history_check,
 {
 "no":"select",
 "yes":"response"
 })

graph_builder.add_edge("select", "code_executor")
graph_builder.add_edge("code_executor", "code_response")
graph_builder.add_edge("code_response", END)
graph_builder.add_edge("response", END);

```

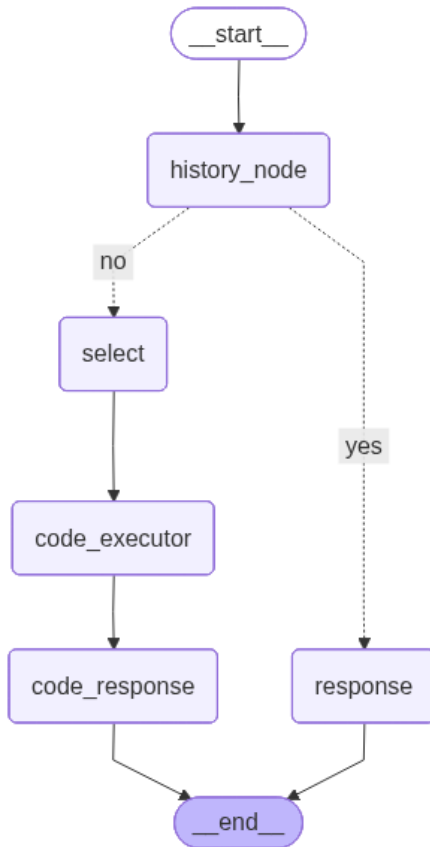
```
메모리 설정 및 그래프 컴파일
from langgraph.checkpoint.memory import InMemorySaver

memory = InMemorySaver()

graph = graph_builder.compile(checkpointer=memory)
```

```
그래프 시각화
가끔 "ReadTimeout: HTTPSConnectionPool(host='mermaid.ink', port=443): Read timed out. (read timeout=30s)"
시간 초과로 그래프 생성에 실패했다는 메시지일뿐 기능과는 관계없으니 진행해도 괜찮습니다.

graph
```



#### ✓ 4. 에이전트 실행 함수

사용자 질문을 받아 전체 워크플로우를 실행하고, 실시간으로 결과를 스트리밍합니다. RunnableConfig로 실행 제한(recursion\_limit)을 설정하여 무한 루프를 방지합니다.

```
사용할 설정값 정의

config = RunnableConfig(recursion_limit=10, configurable={"thread_id": "12"})
```

```
출력 함수 정의
mode = "values" : 상태의 키, 값의 형태로 반환
mode = "updates" : 업데이트되는 값만 반환

def streaming(query, config, mode="values"):

 result = graph.stream({"messages": ("user", query)}, config=config, stream_mode=mode)

 if mode == "values":
 for step in result:
 for k, v in step.items():
 if k == "messages":
 v[-1].pretty_print()
 print("\n\n")
 elif mode == "updates":
 for step in result:
```

```

for k,v in step.items():
 print(f"\n\n=== {k} ===\n\n")
 print(v)

return

```

## 5. 데이터 분석 실행 예제 1

설정된 LangGraph 에이전트를 사용하여 타이타닉 데이터 분석을 수행합니다:

- 생존 여부별 승객 수 분석
- 나이별 승객 분포

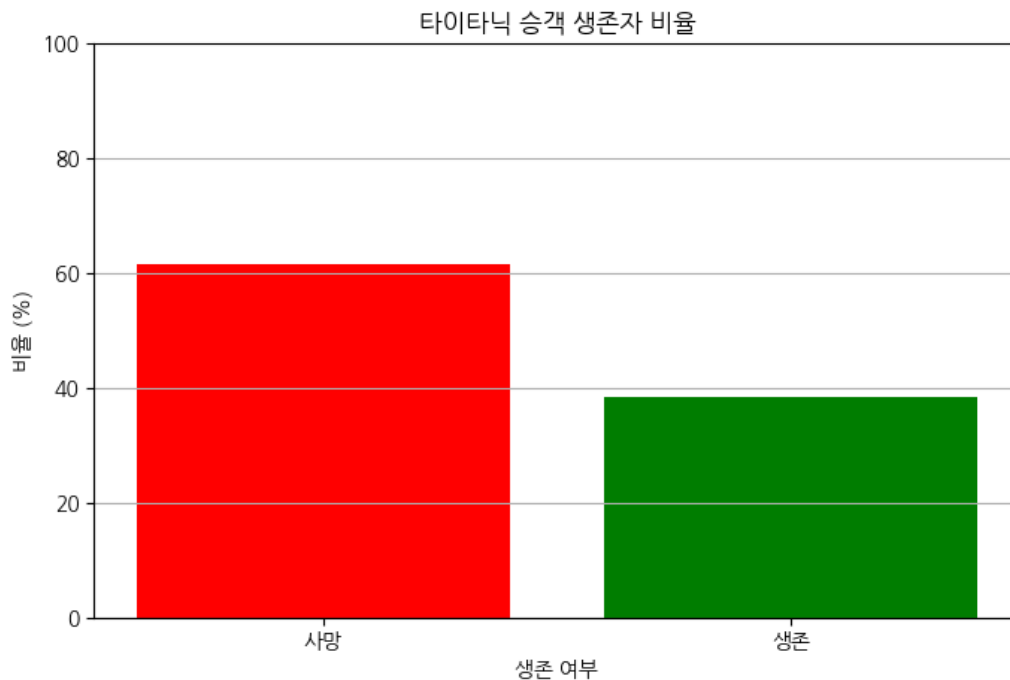
```
streaming("생존자 비율 시각화하고 인사이트 제공해줘", config)
```

===== Human Message =====

생존자 비율 시각화하고 인사이트 제공해줘

===== Human Message =====

생존자 비율 시각화하고 인사이트 제공해줘



===== Human Message =====

생존자 비율 시각화하고 인사이트 제공해줘

===== Ai Message =====

타이타닉 승객의 생존자 비율을 시각화한 결과, 전체 승객 중 생존한 비율과 사망한 비율을 확인할 수 있습니다.

- \*\*생존자 비율\*\*: 약 38% (생존)
- \*\*사망자 비율\*\*: 약 62% (사망)

이 데이터는 타이타닉 호의 비극적인 사건을 반영하며, 생존자와 사망자의 비율 차이가 상당히 큼니다. 이는 구조 작업의 어려움이나 당시 이러한 인사이트는 생존자와 사망자의 특성을 분석하는 데 중요한 기초 자료가 될 수 있습니다. 예를 들어, 생존자와 사망자의 성별, 나이

<Figure size 640x480 with 0 Axes>

```
streaming("아까 내가 질문했던 내용 다시 알려줘", config)
```

```
===== Human Message =====
```

아까 내가 질문했던 내용 다시 알려줘

```
===== Ai Message =====
```

아까 질문하신 내용은 "생존자 비율 시각화하고 인사이트 제공해줘"였습니다. 이에 대한 답변으로 타이타닉 승객의 생존자 비율을 시각화한

```
streaming("아까 물어봤던 숫자들 다 더하면 몇인지 알려줘", config)
```

```
===== Human Message =====
```

아까 물어봤던 숫자들 다 더하면 몇인지 알려줘

```
===== Ai Message =====
```

아까 언급된 숫자는 생존자 비율인 약 38%와 사망자 비율인 약 62%입니다. 이 두 숫자를 더하면:

$38 + 62 = 100$

따라서, 생존자 비율과 사망자 비율을 더하면 100%가 됩니다.

## 6. 데이터 분석 실행 예제 2

설정된 LangGraph 에이전트를 재사용하여 데이터 로드와 도구를 재정의, 올림픽 데이터 분석을 수행합니다.

- 키와 메달 획득 상관관계
- z-score를 활용한 이상치 탐지

올림픽 데이터셋은 여기에서 다운로드 후 사용해주세요!

<https://drive.google.com/file/d/1rmDFdDRJ7wT1PXhLnu6jWyugl3jgmVT/view?usp=sharing>

```
올림픽 데이터 로드 및 도구 재정의
```

```
url = "/content/athlete_events.csv"
df = pd.read_csv(url)
```

```
tool = PythonAstREPLTool(name="python_repl_ast",
 description="A Python shell. Use this to execute python commands. \
Input should be a valid python command. When using this tool, \
sometimes output is abbreviated - make sure it does not look at \
locals={\"df\":df})
```

```
제목, 요약 생성
title, summary = create_title_summary(df)
```

```
===== 제목, 요약 생성 완료 =====
```

```
print("제목 : ", title)
print()
print("요약 : ", summary)
```

제목 : \*\*제목:\*\* 올림픽 선수 데이터셋: 인구통계 및 경기 정보

요약 : \*\*요약:\*\* 이 데이터셋은 3000명의 올림픽 선수에 대한 정보를 포함하고 있으며, 각 선수의 ID, 이름, 성별, 나이, 신장, 체

```
config = RunnableConfig(recursion_limit=10, configurable={"thread_id": "99"})
```

```
streaming("올림픽에는 몇개의 나라가 출전했나요?", config)
```

```
===== Human Message =====
```

올림픽에는 몇개의 나라가 출전했나요?

```
===== Human Message =====
```

올림픽에는 몇개의 나라가 출전했나요?

```
===== Human Message =====
```

올림픽에는 몇개의 나라가 출전했나요?

```
===== Ai Message =====
```

올림픽에는 총 230개의 나라가 출전했습니다. 이는 다양한 국가들이 참여하여 국제적인 스포츠 축제를 이루는 것을 의미합니다. 이 숫자는

```
streaming("가장 많이 출전한 나라는 어디인가요?", config)
```

```
===== Human Message =====
```

가장 많이 출전한 나라는 어디인가요?

```
===== Ai Message =====
```

가장 많이 출전한 나라는 미국입니다. 미국은 올림픽 역사상 가장 많은 메달을 획득한 나라로, 다양한 종목에서 뛰어난 성과를 보여주고 있

```
df["NOC"].value_counts()
```

	count
NOC	
USA	18853
FRA	12758
GBR	12256
ITA	10715
GER	9830
...	...
YMD	5
SSD	3
NBO	2
UNK	2
NFL	1

230 rows × 1 columns

dtype: int64

```
streaming("올림픽에 출전한 선수들의 평균 체중은 얼마인가요?", config)
```

```
===== Human Message =====
```

올림픽에 출전한 선수들의 평균 체중은 얼마인가요?

===== Human Message =====

올림픽에 출전한 선수들의 평균 체중은 얼마인가요?

===== Human Message =====

올림픽에 출전한 선수들의 평균 체중은 얼마인가요?

===== Ai Message =====

올림픽에 출전한 선수들의 평균 체중은 약 **70.7kg**입니다. 이 수치는 선수들의 체중이 다양할 수 있음을 나타내며, 특정 종목이나 체급에

```
df["Weight"].mean()
```

```
np.float64(70.70239290053351)
```

```
config = RunnableConfig(recursion_limit=10, configurable={"thread_id": "996"})
```

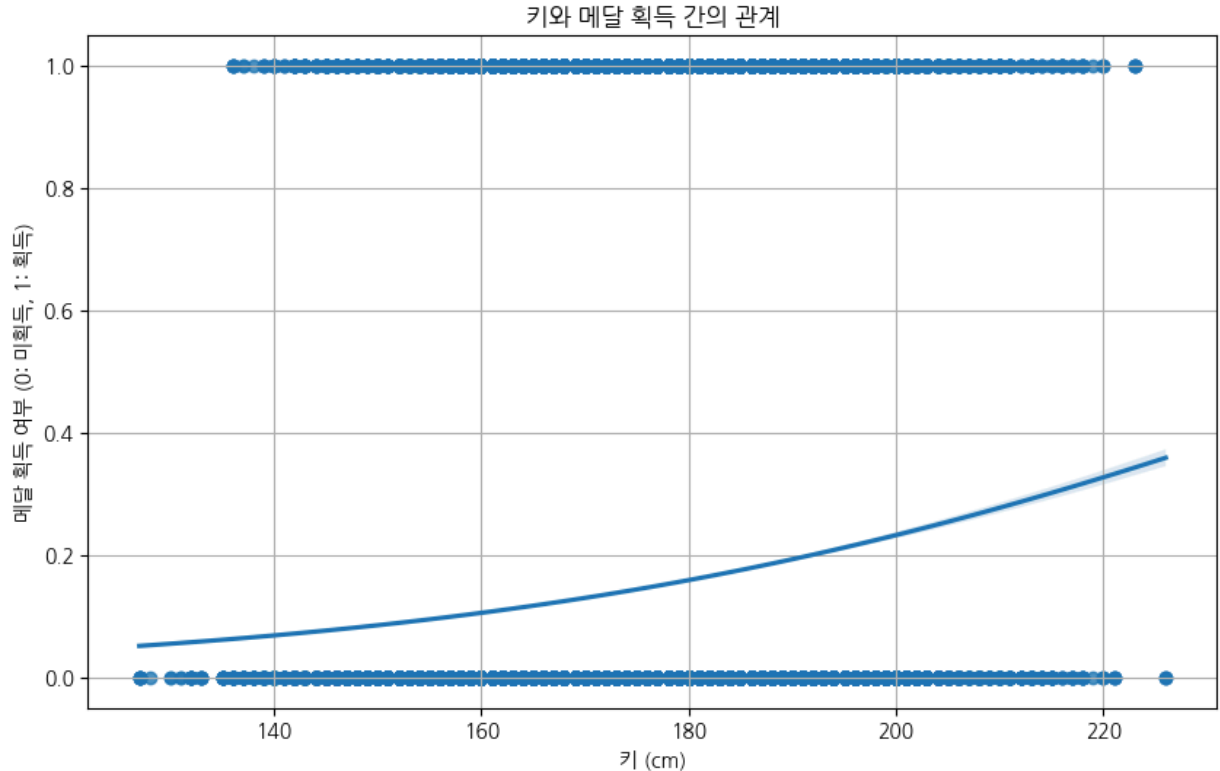
```
streaming("키와 메달 획득과의 상관관계를 보여주세요. regplot 그래프로 그려주세요.", config)
```

===== Human Message =====

키와 메달 획득과의 상관관계를 보여주세요. regplot 그래프로 그려주세요.

===== Human Message =====

키와 메달 획득과의 상관관계를 보여주세요. regplot 그래프로 그려주세요.



===== Human Message =====

키와 메달 획득과의 상관관계를 보여주세요. regplot 그래프로 그려주세요.

===== Ai Message =====

주어진 코드의 결과는 키와 메달 획득 간의 관계를 시각화한 그래프입니다. 이 그래프는 키(단위: cm)와 메달 획득 여부(0: 미획득, 1: 획득)를 나타내며, y축은 메달 획득 여부를 이진 변수로 나타내며, 0은 메달을 획득하지 못한 경우, 1은 메달을 획득한 경우를 나타냅니다. 그래프에서 확인할 수 있는 주요 포인트는 다음과 같습니다:

- \*\*키와 메달 획득의 관계\*\***: 그래프의 경향성을 통해 키가 높은 선수들이 메달을 획득할 확률이 높아지는 경향이 있을 수 있습니다.
- \*\*메달 획득 여부\*\***: y축은 메달 획득 여부를 이진 변수로 나타내며, 0은 메달을 획득하지 못한 경우, 1은 메달을 획득한 경우를 나타냅니다.
- \*\*로지스틱 회귀선\*\***: 그래프에 포함된 회귀선은 키가 증가함에 따라 메달을 획득할 확률이 어떻게 변화하는지를 나타냅니다. 이 선은 키가 증가함에 따라 메달 획득 확률이 증가하는 경향을 보여줍니다. 이러한 인사이트는 선수 선발, 훈련 프로그램 개발, 그리고 스포츠 과학 연구에 중요한 정보를 제공할 수 있습니다.

<Figure size 640x480 with 0 Axes>

```
config = RunnableConfig(recursion_limit=10, configurable={"thread_id": "999"})
```

```
streaming("키와 체중, 그리고 메달 획득과의 상관관계를 보여주세요. 산점도 그래프로 그려주세요.", config)
```